ShipGuard: A Fragility-Aware Shipment Monitoring Node

Seojin Hong

1. Project Description and Goal

Fragile, high-value goods such as glass, precision instruments, and electronics break when shocked or left tilted during transport. Passive shock labels give no alert or data until arrival, and data loggers are analyzed only after delivery, so nobody can step in when the risk rises, and damage leads to evidence disputes among the shipper, carrier, and customer.

ShipGuard is a battery-powered node placed inside the box. It computes a risk score at the edge based on the product's fragility profile (glass, electronics, robust goods), sounds local alarms by risk level, and records the driver's acknowledgment (ACK) and response time as evidence. The industry is logistics, and the scope is the last mile in one metropolitan area: regional warehouse, delivery van, retail stores. The realistic target is the **premium (handle-with-care) delivery service** for fragile, precision, and high-value goods, because the sensors, modem, and SIM make the node too costly for every product; it pays off only where damage is costly.

It makes inspection more efficient (only boxes whose risk increased are checked), cuts the cost of damage, returns, reshipping, and disputes, leaves objective evidence of shocks, alerts, and ACKs, and reduces discarded goods and reshipping trips.

Similar products exist (Lansmont SAVER AM, Tive, with product sensitivity monitoring). I do not claim to be better, but as a learning prototype I designed and measured per-profile edge risk scoring, alert rules that clear by alert type, and a closed loop to the driver's ACK and response time.

The **goal** is not to promise that damage will be prevented, but to **quantify the risk and shorten the time until a person intervenes**.

2. The "Things": Smart Object Characteristics

Table 1. Summary of smart object characteristics

Item	Design assumption
Quantity, other things	About 2,000 returnable nodes for 1 warehouse, about 60 vans (about 25 nodes each), and about 150 stores; the prototype is 1 node. Other things: driver's smartphone app (gateway), carrier's LTE-M network, cloud (MQTT broker, CoAP receiver, database), role-based dashboards.
Constraints	Box-sized and shock-proof, with microcontroller-level memory (UNO: 2 KB SRAM), so only features are computed at the edge; security is needed for location, cargo, and driver data.
Power, energy saving	Rechargeable Li-ion battery, charged at each collection; target at least 14 days per charge and 3 months on standby. Sleepy end device with duty cycle; BLE advertising only; LTE-M PSM/eDRX; no energy harvesting.
Deployment	Inside the product box, across a metro area within about 50 km (warehouse, van cargo area, store dock, brief outdoor handling); stationary nodes gather in the warehouse; nodes are tens of cm to a few m apart.
Mobility	Mobile; a few hours to a day from dispatch to store receipt, about one round trip per day.
Gateway, topology	Van: driver's smartphone (about 5–15 m); warehouse: LTE-M directly to the cell tower (Table 4). Star topology.

Item	Design assumption
Data	Only summaries (about 24 B every 60 s in the van, 15 min in the warehouse) plus immediate events, with priority L3 > L2 > ACK/settings > summary; no raw data, since location, cargo value, and driver data are sensitive.
Analysis	Edge (node): risk score and alerts; fog (phone): instant alerts, store-and-forward; cloud: fleet analysis and reports.

Why inside the box, not outside? The node sits in the cushioning next to the product. First, damage depends on the shock the product receives, not the box surface, and the profile limits (e.g., 4 g for glass) are product-based; an outside sensor would read too high and cause false alarms. Tapping the bare board on a desk exceeded ±16 g, but drops inside a cushioned box were mostly 5–12 g (a few hard drops still saturated). Second, the light sensor detects opening only inside the dark box. Third, tilt is measured relative to the product, set together at dispatch. Fourth, outside the box it can fall off, break, or be removed or swapped; inside, only opening the box reaches it. The trade-off is a weaker signal and a muffled buzzer, so the driver's phone stays close in the van and its notification is the main alert channel (used only as much as needed, so the driver is not disturbed while driving).

3. Smart Object Prototype

The prototype is built with an Arduino UNO R3, the Arduino Sensor Kit, and an additional PIR motion sensor. It uses 7 sensors and 3 actuators.

Table 2. Sensors and actuators

Component	Role in ShipGuard	In the real product
3-axis accelerometer (LIS3DHTR)	Shock peak g and ΔV; tilt from gravity. 400 Hz, ±16 g	Same role
Light sensor	Detects the box being opened	Same role
Temperature and humidity (DHT20); barometer (BMP280)	Environment record	Same role
Potentiometer	Selects the product profile	*Emulated:* clerk scans the node's QR code in the app to set the profile
Button	Driver ACK	*Emulated:* confirm tap in the driver app
PIR motion sensor	Driver arrival	*Emulated:* driver's phone coming close over BLE
Buzzer	L2 beeping, L3 continuous tone	Local alarm; the driver's phone notification is the main channel
LED; OLED display	LED on during an alert; OLED shows profile, risk score, tilt, level, ACK prompt	Local indicators

I used the PIR to emulate the driver's phone coming close because a PIR inside a closed box cannot see anything.

Execution logic. The code is about 800 lines, and the risk calculation, state machine, and frame handling are separated into C modules that do not depend on Arduino and were unit-tested on a PC (10 passed). The logic is as follows.

1. Read the acceleration every 2.5 ms (400 Hz, so millisecond shocks are not missed). If the acceleration minus gravity exceeds 2 g, treat it as a shock event and compute the peak g and velocity change (ΔV). The ratio to the profile's shock

limit (GLASS 4 g, ELECTRONICS 8 g, ROBUST 14 g) is the shock score, and if the sensor saturates, it is treated as the most dangerous case.

2. If the box stays beyond the profile's tilt limit (30°, 45°, 60°), the tilt score rises in proportion to how long it lasts. Beyond 1.5 times the limit, it goes straight to the L2 level.
3. The risk score is the larger of the two scores. 40, 70, and 100 or more are L1, L2, and L3. Going down uses hysteresis: the score must stay below the threshold for 5 seconds.
4. At L2 or higher, the node enters the alert state; the buzzer, LED, and OLED operate, and the buzzer keeps sounding until the driver ACKs.

Alert rules. I designed two kinds of alerts. Tilt is a "condition", so if the box is set upright before the driver ACKs, the alert clears by itself. A shock is an "event" and the product may already be damaged, so it does not clear until someone ACKs and inspects. The ACK only silences the buzzer; the alert display stays until the risk actually goes down, and if the risk rises further after the ACK, it sounds again. If the PIR detects the driver during an alert, the OLED shows "PRESS BTN TO ACK", and the dashboard computes the time from alert to arrival to ACK.

Interactions among sensors, actuators, and the dashboard. When a shock exceeds the limit, the buzzer, LED, OLED, and an alert frame go out together. From the PC, I can send ACK, profile change, report interval change, and new shipment commands (event-driven and infrequent).

Data sent to the computer. Every message is one frame (start byte, length, type, sequence number, up to 24-byte payload, CRC-8). The payload carries the risk score, level, state, profile, latest shock peak g and ΔV, tilt, temperature, humidity, pressure, and flags (opened, driver present, saturated, ACK pending); shocks, level changes, ACKs, and driver arrival get their own frames. Each field maps to a cause of damage (shock, tilt, environment, opening).

4. Communication Design

I designed but did not implement the communication stack, because the Arduino is connected only by wire; Table 4 shows which radio replaces the prototype's USB serial link in each segment, and the last row of Table 8 maps the prototype to the deployed system.

Architecture. I use the Simplified IoT Architecture from class (Figure 1). The node is the Things layer and the edge; the network layer covers access (BLE 5, LTE-M), gateway and backhaul (driver's phone or carrier network), transport (MQTT, CoAP), and management (node–shipment mapping, settings); the application layer serves drivers, stores/customers, and shippers. I chose it because it maps directly onto edge–fog–cloud and lets me justify each layer's technology separately.

Data flow and topology. (1) In the van, normal operation: node → BLE advertising → driver's phone → LTE/5G → MQTT broker → dashboard. (2) Waiting in the warehouse: node → LTE-M (15-minute summary) → cell tower → cloud. (3) If L3 occurs in the van and the driver's phone does not respond, the node sends the alert directly over LTE-M. Both paths are star: in the van, nodes are close and the cargo layout keeps changing, so mesh routing costs more than it helps, and in the warehouse each node reaches the tower in one hop.

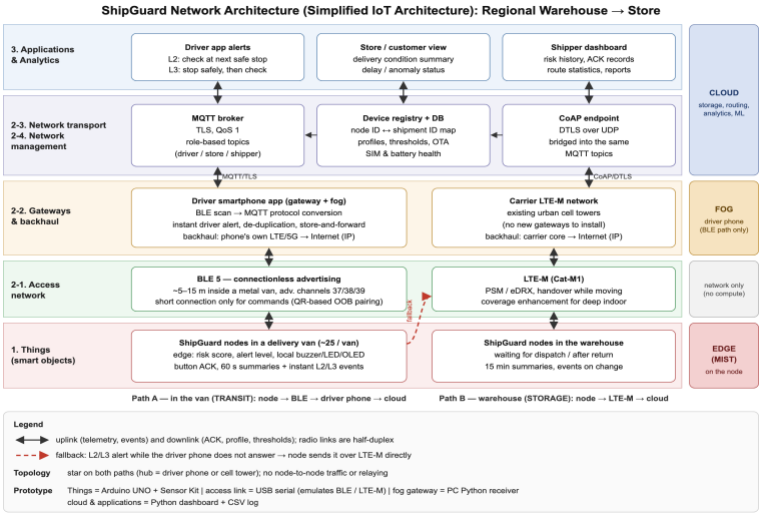


Figure 1. ShipGuard network architecture

Table 3. Communication constraints and design responses

Constraint	Design response
Obstacles: metal van body and racks, water-filled goods, tunnels and underground docks	Short in-van BLE link; LTE-M coverage enhancement; store-and-forward
Interference: BLE shares 2.4 GHz with Wi-Fi; about 25 nodes advertise in one van	Advertising channels avoid Wi-Fi 1/6/11; random advDelay; repeated events de-duplicated by sequence number; LTE-M is licensed
Range: BLE about 5–15 m; no phone in the warehouse	BLE only in the van; LTE-M (km range) elsewhere
Energy: LTE-M transmit current (hundreds of mA) far above BLE (a few mA)	BLE advertising normally; LTE-M sleeps with PSM/eDRX
Constrained device: Class 0 node (RFC 7228)	Phone handles protocol translation and TLS; LTE-M uses CoAP over UDP
Data rate, latency: 24–32 B messages; L2/L3 must reach the driver within seconds	Both radios suffice; the phone (fog) alerts the driver without a cloud round trip
Mobility: nodes move across the city	LTE-M handover (NB-IoT excluded)
Cost: modem and SIM for about 2,000 nodes	About 10 KB/day or less per node fits low-volume IoT plans
Security: sensitive data; open advertisements; replay; route tracking	AEAD and sequence numbers; rotating BLE address (RPA); SIM authentication and DTLS; role-based cloud permissions
Other: clock drift, updates, duplicate events, SIM management	Time sync; OTA updates; sequence-number de-duplication; per-node SIM management

Access technologies. The node uses only two radios, BLE 5 and LTE-M (Table 4), because the van (phone always present, short distance, power matters) and the warehouse (no phone, deep indoors) are different, and one technology cannot satisfy both. BLE 5 is low power and supported by smartphones by default, so no new gateway needs to be installed in vehicles. LTE-M uses the cell towers that already exist in the city, so no gateway installation is needed; it reaches deep indoors with coverage enhancement and supports handover while moving. NB-IoT was excluded because its mobility is limited. LoRaWAN and NFC were removed after the poster feedback (Table 8).

Table 4. Wireless technology for each segment

Segment	Node radio	Gateway	Backhaul	When used
In the van	BLE 5 advertising	Driver's smartphone app	Phone's LTE/5G	Normal operation during transport
Warehouse	LTE-M	None (direct to cell tower)	Carrier core network	Waiting for dispatch, after return
In the van, phone not responding	LTE-M	None (direct to cell tower)	Carrier core network	Fallback for urgent L2/L3 alerts

Identity, IP, and transport. Because 2,000 nodes are reused, scanning the node's QR code at dispatch registers the node–shipment mapping in the cloud; the node then sends only encrypted advertisements, the driver's phone sees all 25 nodes by scanning, and a brief connection is used only for commands. On the BLE path IP starts at the phone, while an LTE-M node gets an IP address from the carrier; with a one-hop star, no node-side routing protocol is needed. The links are half-duplex and mostly upstream. The phone uses MQTT over TLS (QoS 1), which fits role-based topics and guarantees delivery, and LTE-M nodes use CoAP over DTLS, since UDP suits sleeping nodes; the system is client–server, because all stakeholders need one record merged from both paths; no P2P, since node-to-node links only cost power.

Scalability and ethics. Security and privacy measures are in Table 3. Using existing cell towers means no new gateways as nodes or cities are added. Records support drivers rather than punish them, and alerts say "inspect at the next safe stop" so they never require action while driving.

5. Results: Data Analysis

Collection method. The Python receiver on the PC receives the serial frames, checks the CRC, and saves them to CSV. The Python dashboard redraws from this CSV every second. For the demo, I shortened the report interval to 1 second.

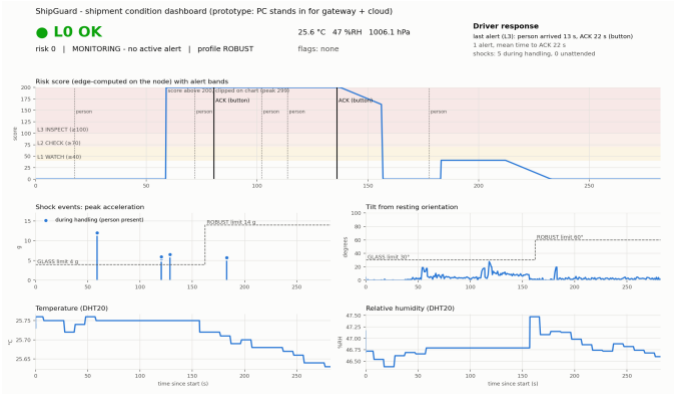


Figure 2. Dashboard: risk score with L1–L3 bands and ACK/arrival markers, shock peaks with profile limits, tilt, temperature and humidity, and a driver-response summary

Table 5. Measured results

Test	Result
Accelerometer sampling (400 Hz setting)	393 samples read in 1 second; no loss at 400 Hz or below
GLASS, box drop	Peak 12.0 g, ΔV 3.9 m/s → score 299, L3
Driver response (same alert)	Arrival detected after 13 s, button ACK after 21.5 s; a later 5–6.5 g drop re-sounded the alarm

Test	Result
ROBUST, drop from a similar height	Peak 5.7 g → score 41, L1 (the same shock under GLASS would compute to L3)
GLASS, tilt	Set upright: L2 cleared automatically after about 6 s without ACK; kept tilted: L2 rose to L3 after about 30 s
Serial link	0 CRC errors and 0 missing frames across several runs
Memory	Flash 21,986 B (68%), SRAM 1,092 B (53%), free SRAM at runtime 887–914 B

Analysis. The profile changed the level for the same shock: the 5.7 g shock that was L1 under ROBUST would score 143 (L3) under the GLASS limit (4 g). Cloud machine learning could detect abnormal handling and recalibrate profiles from damage claims, because that needs fleet-wide history; the edge and the phone (fog) handle fast, low-power alerts.

Limitations. These results come from a small number of trials during the demo and functional checks, without a controlled drop experiment, and the thresholds and profile limits are example values not calibrated with damage data. Also, the PIR detected a person even at the moment of the drop (range of several meters), so separating "unattended shock (vehicle)" from "shock during handling" could not be shown.

6. Conclusions

6.1 What I Learned

I learned that design and implementation are different: alert logic that looked right on paper failed on the real board (Table 6), which taught me why "condition alerts" and "event alerts" must be separated. I also learned that narrowing the scope is what makes technology choices justifiable: at a global scope I could not say where LoRaWAN gateways would go, but in one metro area the answer was clear, the existing LTE-M cell towers.

6.2 Questions for Further Research

How well does the risk score predict real damage, and how should profile thresholds be calibrated? Controlled drops with real goods and damage claim data could answer this. What are the BLE advertising loss rate and alert delay in a real van, and how fast is the LTE-M fallback? How much battery life would accelerometer-interrupt wake-up add?

6.3 Main Issues and Challenges, and How I Overcame Them

Table 6. Main issues and how I overcame them

Issue	How I overcame it
The poster's design was impractical at a global scope and lacked the architecture and communication constraints.	Narrowed the scope and redesigned the communication plan, confirmed by the professor (Table 8).
Just tapping the bare board saturated the ±16 g accelerometer.	Fixed the range at ±16 g and treated saturation as the most dangerous case (L3).
Alert logic bugs: the alarm turned off early, an ACK pressed in advance, a PC profile change overwritten one second later.	Fixed the state machine with PC unit tests (rules in Section 3); the potentiometer now applies only when it is moved.
The risk score did not reset from 999 after a shock.	A design choice: a saturated shock's size is unknown, so 999 is held for 30 s and decays by 2 per second so it is not forgotten too quickly; ARM resets it.

Issue	How I overcame it
The PIR's range was too wide to create a "no one present" state.	Increased its hold time to stop flickering and hid the PIR under the Sensor Kit in the demo.
The Grove accelerometer exposes no interrupt pin, so low-power wake-up could not be built.	Explained low power with a datasheet-based design and specified sensor-interrupt wake-up for the real product.

6.4 Response to Peer Review Feedback

Table 7. Peer review feedback and responses

Feedback	Response
Basis for the 40/70/100 thresholds and per-profile calibration	The values are examples, but per-profile limits already give different levels for the same shock (Table 5); calibration with controlled drops is the next step.
Validate alert accuracy and false alarm rate with real drops	Only functional-check drops were done; controlled experiments are future work.
Show saturation as "out of range"; why use the maximum of the shock and tilt scores?	Saturation is flagged and treated as L3 (a wider-range sensor is future work); the maximum is used because the two risks are different causes, and the more dangerous one drives the response.
Unclear BLE/LoRaWAN/LTE-M switching; the phone is an unreliable gateway, so use a fixed one	Radios reduced to BLE 5 and LTE-M (Table 4); if the phone does not respond, the node uses LTE-M, avoiding a fixed gateway in every vehicle.
BLE replay, lost-node key revocation, node–shipment permissions; remote settings to a broadcasting node	AEAD and sequence numbers block replay; the cloud registry handles role-based permissions and key revocation; settings use brief connections (Section 4).
How long before escalating without an ACK?	Designed as 300 s (L3) and 600 s (L2) before notifying the shipper; not implemented in the prototype.

Table 8. Response to instructor feedback on the poster

Feedback	Change made
Global scope makes LoRaWAN impractical; reduce the scope	Scope reduced to one metropolitan area, regional warehouse → stores; air and long-haul transport removed
Wireless technologies may not be feasible at this scale	LoRaWAN replaced by LTE-M on existing carrier towers, so no gateway deployment is needed; LTE-M chosen over NB-IoT for mobility
Missing network architecture	Defined the Simplified IoT Architecture: three functional layers (four sublayers in Layer 2) plus edge/fog/cloud tiers, a function for each layer, three data-flow scenarios and an architecture figure
Missing communication constraints	Added a constraint table (obstacles, interference, range, energy, constrained devices, data rate, mobility, cost, security) with a design response for each
Several technologies mentioned but not clearly specified	The node uses exactly two radios, BLE 5 and LTE-M, shown in a segment × technology table. NFC removed (replaced by QR code + BLE); one protocol per path (MQTT/TLS and CoAP/DTLS)
General: state which wireless technology replaces the wired link	The Arduino prototype keeps a wired USB serial connection to the computer. In the deployed product, BLE 5 replaces this wired link inside the van (node → driver's smartphone), and LTE-M is used in the warehouse or when no phone responds (node → carrier cell tower). In the prototype, the PC's Python receiver stands in for the driver's phone gateway (fog), and the Python dashboard stands in for the cloud and application layers.